

≡ R4R: Guided Tutorial



To navigate slides you can either use the arrows in the bottom right corner or the left and right buttons on your keyboard. Follow along on the PC lab screen for this part of the workshop.

≡ Introduction

Workshop Aims:

- Provide an introduction to R and RStudio in preparation for PS2010 Psychological Research Methods.
- Understand the layout and key features of RStudio.
- Understand the basics of using R code.
- To begin working with data in RStudio and understand key terminology.



About this Session

- The first part of the workshop involves a guided tutorial where you should follow along on screen.
- Raise your hand at anytime if you need help, have a question, or get stuck—someone will come over to assist you.
- We are not able to offer technical support with R/RStudio on your personal device. For today, please use the campus PCs.
- The second part of the workshop involves a practice activity to help you feel more confident with R before lecture begin next week.



R and RStudio



- Rrrrr is not only the noise a pirate makes...
- R is also a programming language. You'll also use RStudio.
- R and RStudio are not the same thing:
 - `R` = the language
 - `RStudio` = the program.

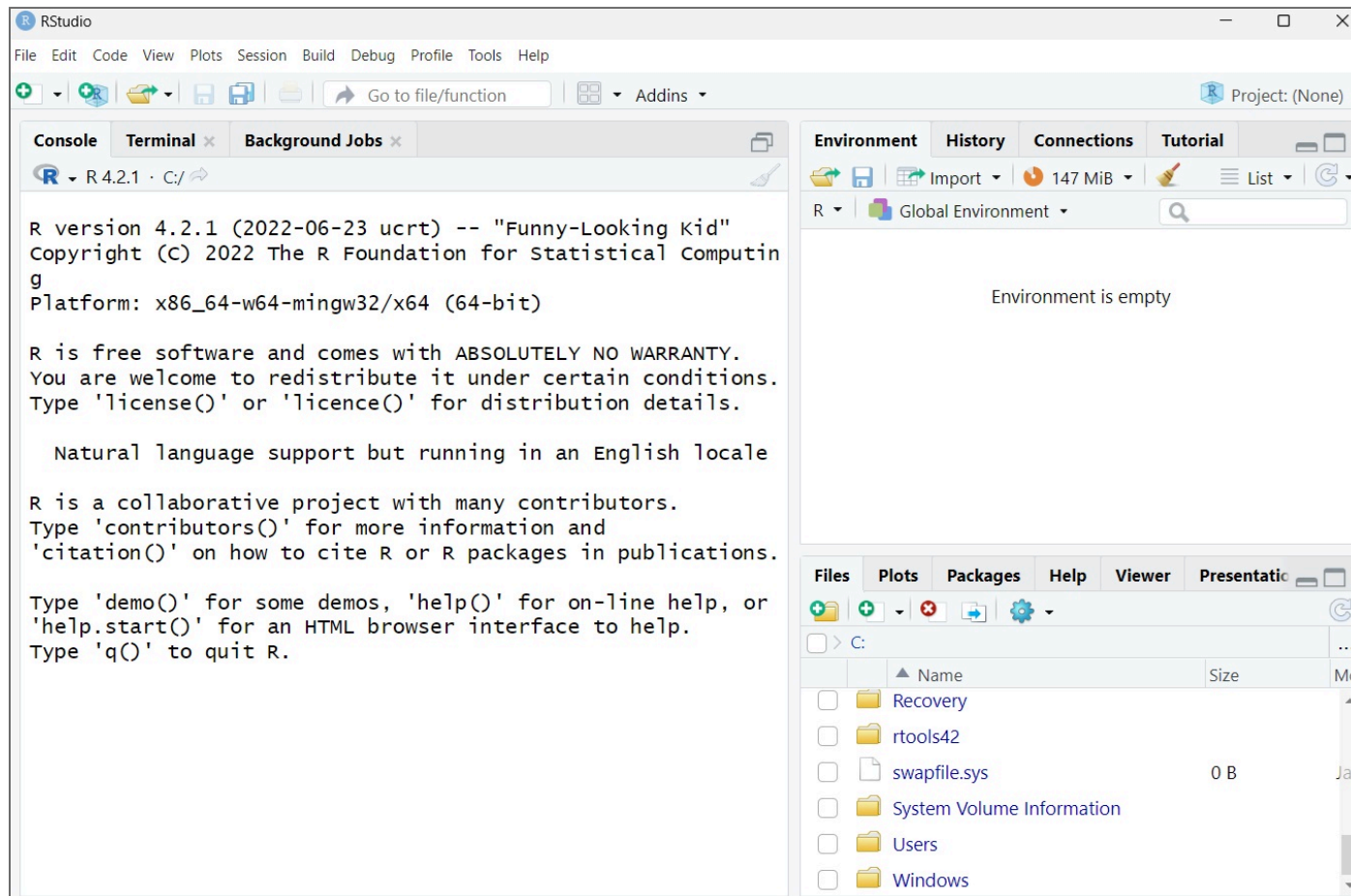
≡ Open RStudio (1)



- Search for “RStudio” on the PC and identify the logo (above).
- You want to make sure you open RStudio and not “R” which also exists on the computer. R just runs in the background.
- A common error is to open the language and not opening the “Studio” program. Make sure you open RStudio.

Open RStudio (2)

When you first open RStudio it should look like this:



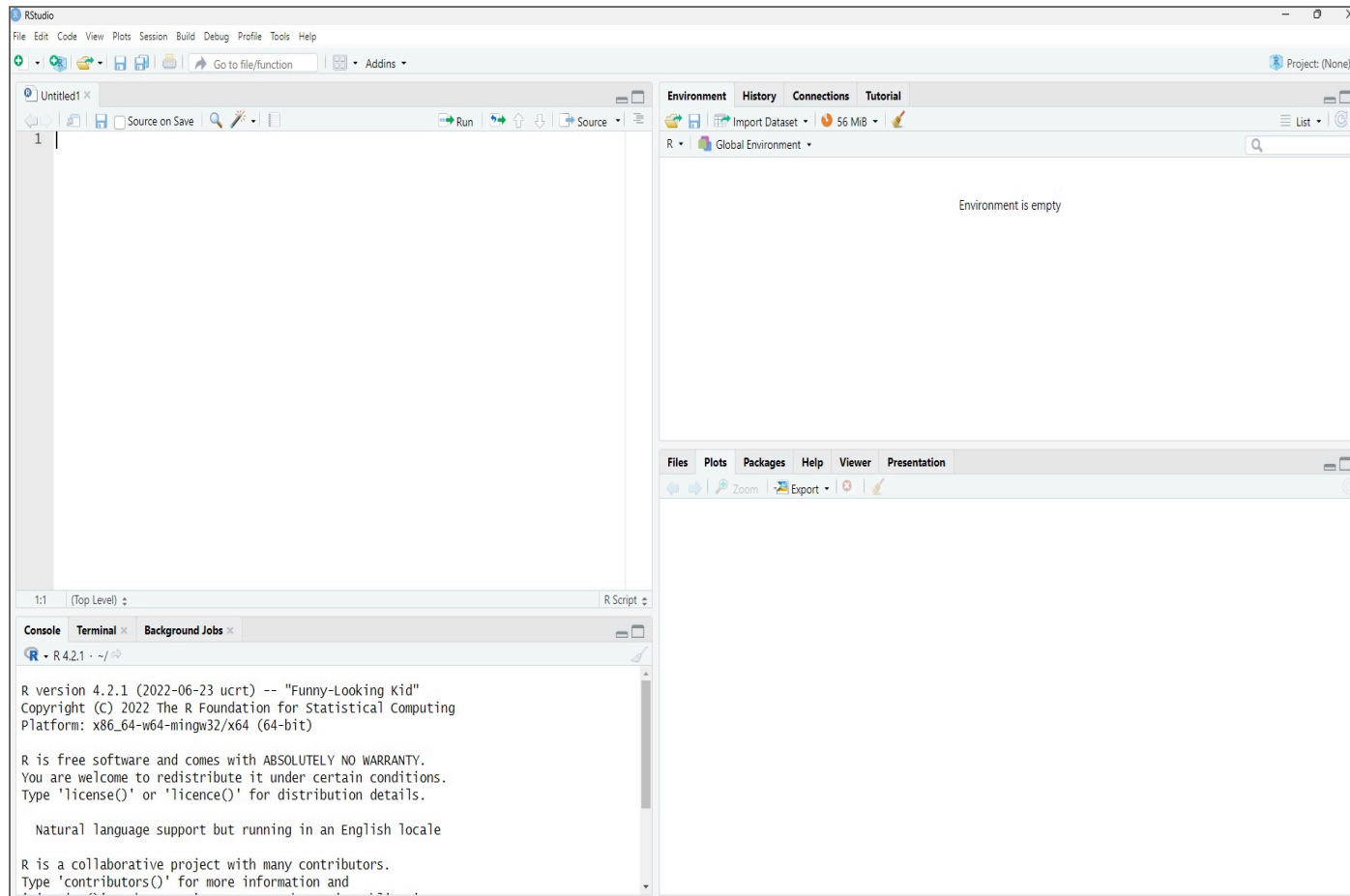


RStudio Layout (1)

- When RStudio opens you'll see **three** panels. We actually want a fourth open.
- Open a new script by going to: `File > New File > R Script`
- You can also use:
 - `CTRL + SHIFT + N` (windows)
 - `CMD + SHIFT + N` (mac)

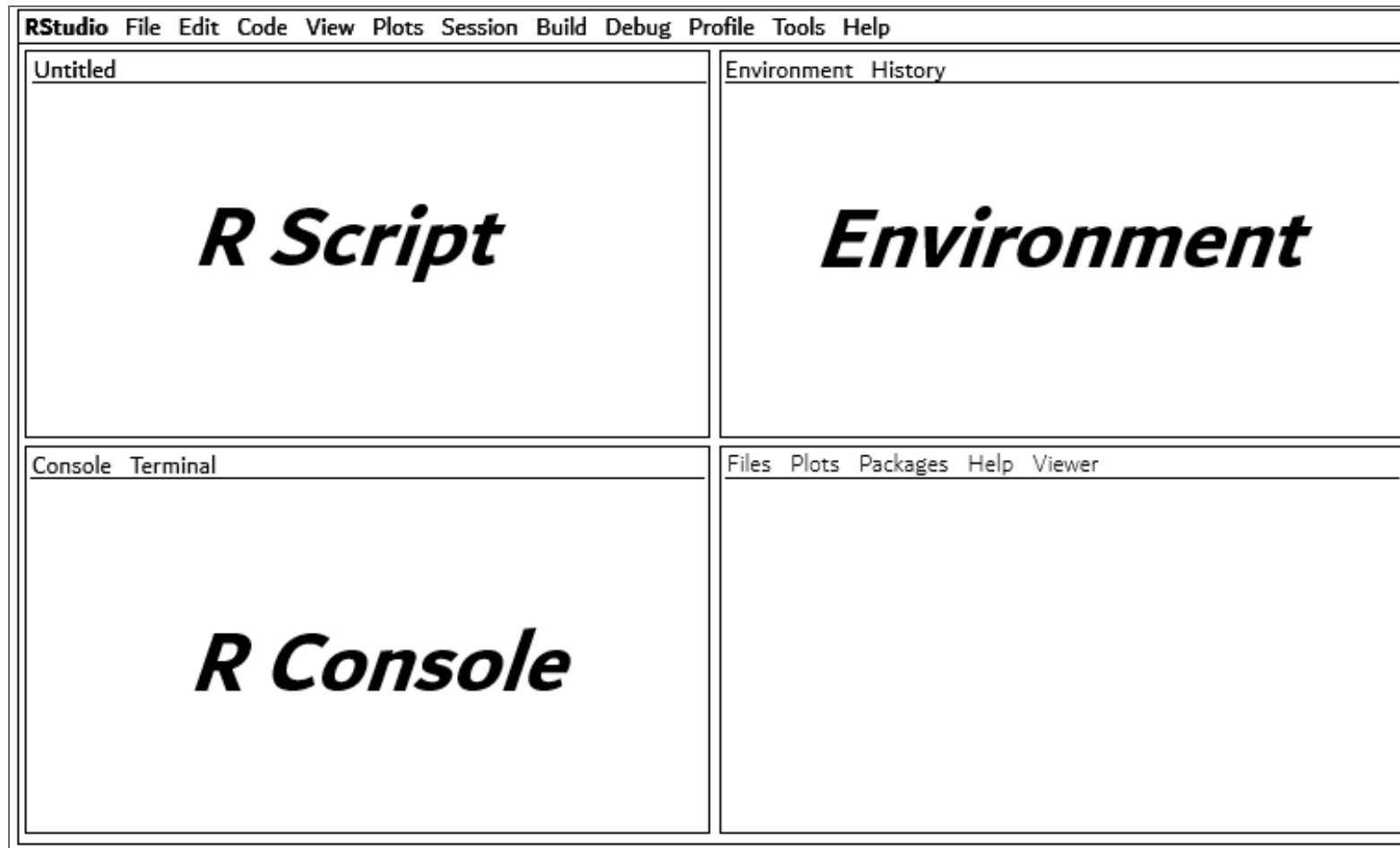
≡ RStudio Layout (2)

Now your RStudio should look like this. There are **four** panels:

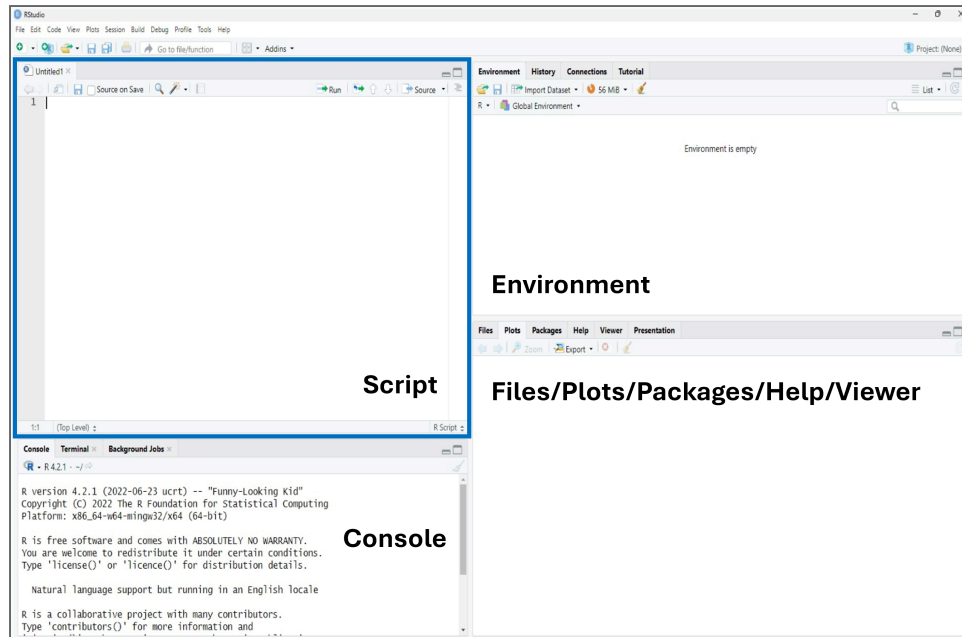


≡ RStudio Layout (3)

You need to learn what each panel does. We will go through them one by one.



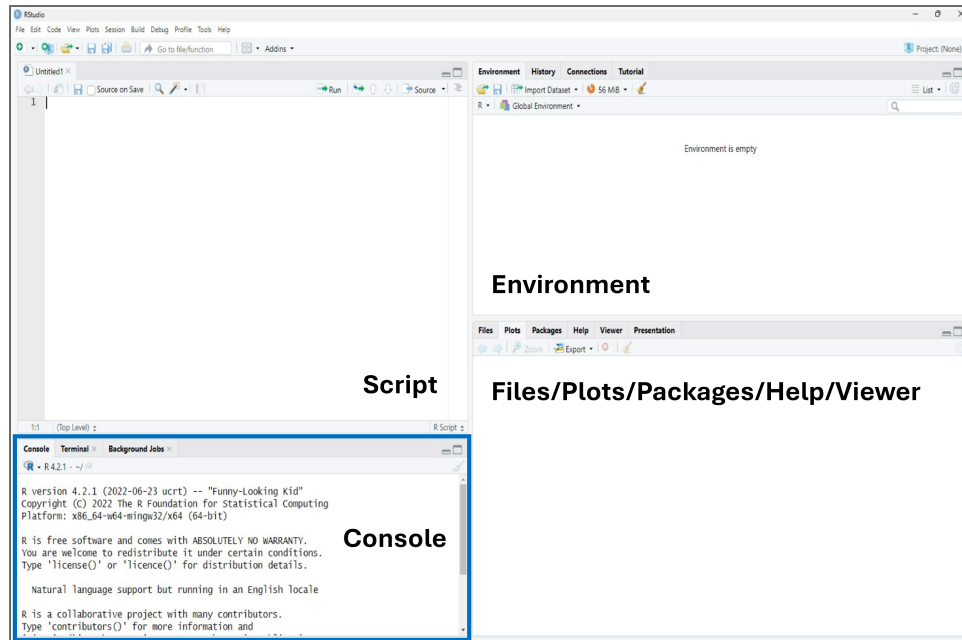
≡ Script Panel



- The R script is the top left and is an editable space where you can type, enter, and run your code. You can save and share your R script, a bit like a Word document.



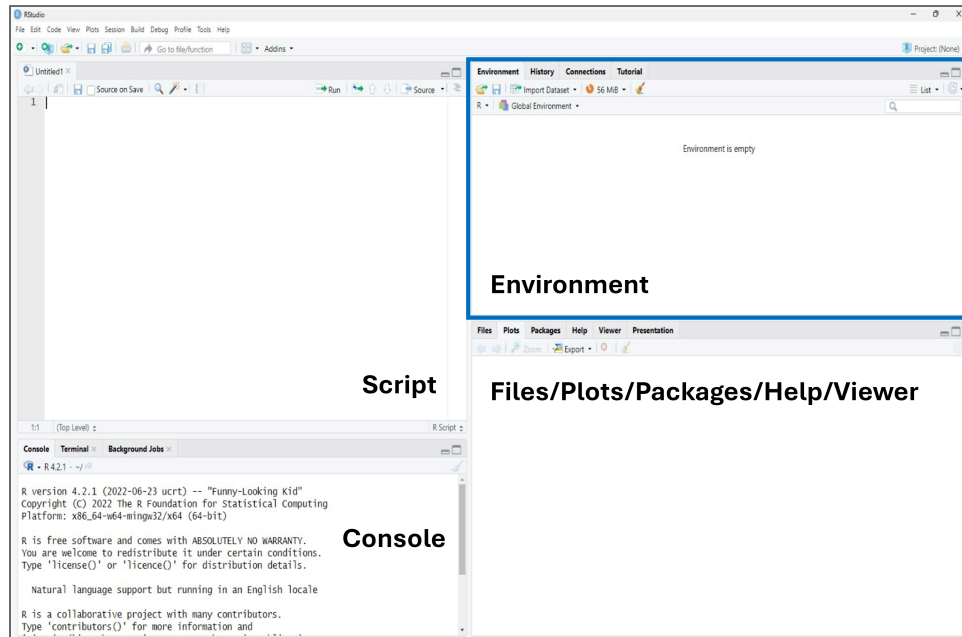
Console Panel



- The R console is the bottom left panel. Think of this as a “results” panel. This is where you can read the results of your code (or an error message if your code did not work!).
- Avoid typing any code into this panel. Code entered directly into the console is not saved in your script.

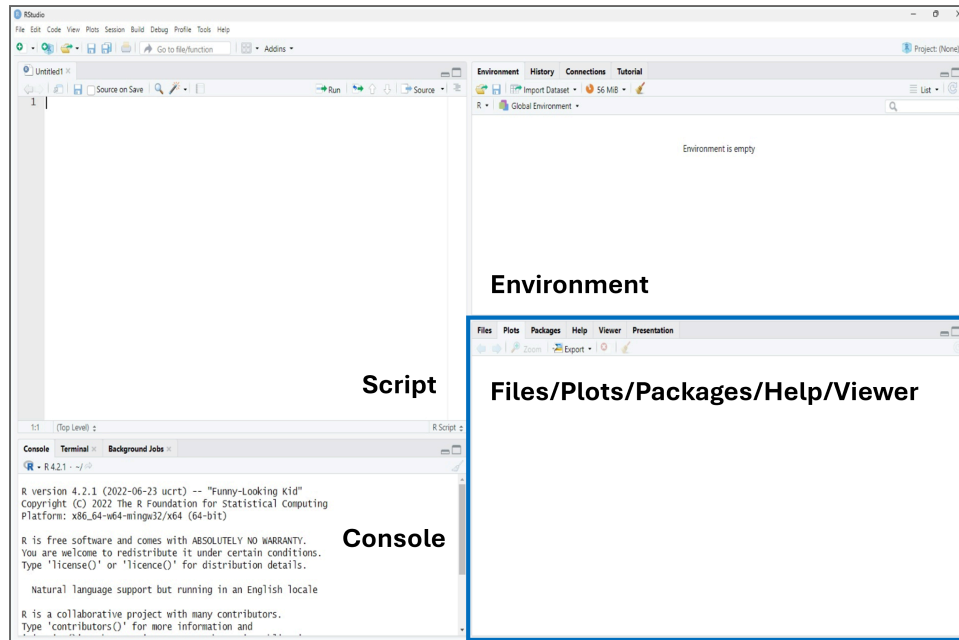


Environment Panel



- The environment panel is the top right panel. It can be helpful to think of this as a bit like a memory space where RStudio will keep hold of any objects/data/information that you ask it to. You'll learn more about saving things to the environment later in this chapter.

≡ Miscellaneous/Output Panel



- The bottom right panel is what I call the miscellaneous panel. It has a few different purposes that include viewing files, plots, packages, or finding help.
- Also known as Files/Plots/Packages/Help/Viewer panel.

≡ RStudio Layout Summary

- As you progress through this workshop and PS2010 this year, you'll learn more about what these panels can do.
- A key take home message is that you need **four** of them and that you enter and save your code in the script (top left) panel.

≡ Writing Code (1)

- Coding sounds scary. However, just think of it as a series of instructions.

```
'stand up', and then,  
'walk towards the window' and then,  
'return to your seat'
```



- Like with any language there are rules and a structure.
- It can be a bit quirky to begin with, even the smallest mistakes can change the meaning.

≡ Writing Code (2)

- **R won't guess what you mean. It needs instructions to be written exactly.**
- A typo or incorrectly placed symbol could mean your code does not work—proof read it carefully and take your time.

PUNCTUATION SAVES LIVES

Let's eat grandma!

or

Let's eat, grandma!





R Code Basics (1)

- In RStudio find the script panel (top left).
- Enter the following code:

```
date()
```



- When ready use your keyboard
 - **CTRL** + **ENTER** (windows)
 - **CMND** + **ENTER** (mac)
- Check the console panel (bottom left) and you should see today's date as the result!



R Code Basics (2)

- Congratulations! You've run your first bit of code. Now what's next?
- RStudio is a bit like a very fancy calculator.
- Try and run this code:

```
33 + 67
```



- Or something more complex:

```
12345*54321
```

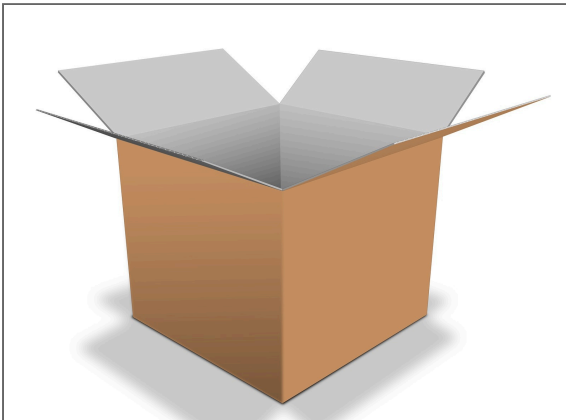


≡ R Code Basics (3)

- In case you have wondered the answer to 12345×54321 it is 670592745.
 - Note that we use an asterisk `*` for multiplication.
 - You can use `/` for division.
- Now you should begin to understand the very basics of running code (giving a command or instruction) and then checking the console to see the output, results, or answer.

≡ Objects and the Environment

- An object is a key feature of RStudio. An object is a bit like a box.
- We can store or place something in our box and then place it in a storage unit, like a warehouse (this analogy will make sense soon!).





Creating an Object (1)

- In RStudio we can assign a value or data to an object. However, you will need to create and name your box.
- We give our box a name so that we can find and retrieve it again later.
- Let's work through an example.

≡ Creating an Object (2)

```
box <- "chocs"
```



- `box` is the name of the object ready for something to be placed inside. You could call a box anything you like, I could have called it `myobject` or `data` if it contained some data.

≡ Creating an Object (3)

```
box <- "chocs"
```



- Next there are two individual symbols, which when combined become an assignment operator. It is essentially an arrow `<-` made up of a less than sign `<` and a dash `-`
- This means anything on the right side of the arrow will be placed into the object (or our newly named box).

≡ Creating an Object (4)

```
box <- "chocs"
```



- "chocs" is what I've put in the box. A box of chocolates.
- It is text information (rather than numbers or data) and so it is in speech marks in our code.
- Run the code now and then check the environment panel (top right).

≡ Creating an Object (5)

- If you look at the top right panel (called the environment), you should spot that an object has appeared called `box`. That's your newly created object.
- The environment panel acts a bit like a warehouse or storage unit for our boxes (see, I told you the analogy would make sense...).
- When conducting data analysis, you may end up with quite a few objects stored in the environment.

≡ Why Do We Need Objects?



- Objects allow us to handle our data efficiently, without them, we would have to write everything from scratch every time.
- Objects save you time and effort by letting you:
 - Store data and information in one place.
 - Reuse your work.
 - Keep your code clean and organised.

≡ Creating More Objects

- Almost anything can be committed to an object in RStudio. For example, run and adapt the code below. Change “Luke” to your own name! Then check your environment to see these additional objects.

```
my_name <- "Luke"  
answer <- 12345*54321  
pi <- 3.14
```





print()

- You can check what is in a box easily by running this code:

```
print(box)
```



- Change `box` to the name of the other objects: `my_name` , `answer` , `pi` and check the console each time you run the code.

≡ Naming Objects (1)

- Try to keep object names short.
- Often you will need to refer to them in your code, and typing out long names is time consuming.
- Always use lowercase letters and use an underscore `_` if you need a space (or just avoid spaces if you can).
- For example:

```
name <- "Luke"
```



- `name` is shorter than `my_name`



Naming Objects (2)

```
ans <- answer
```



- `ans` is shorter than `answer`
- You might notice here you've put an existing box into a new box called `ans`.

≡ coDe iS CasE SenSiTive

Try running:

```
print(Name)
```



- This won't work because R is not expecting an uppercase **N**
- Use this instead:

```
print(name)
```



≡ When R Doesn't Understand

- **Errors will happen!**
- **R is very literal, and error messages often give you clues.**
- Try this:
 - What was I trying to do?
 - What is R telling me?
 - Check what R mentions in the error message.

≡ Functions and Arguments (1)

object name

< -

function

(argument)

- Coding in R is mostly built around using a series of functions and arguments—both work together to carry out your instruction(s).
- You have actually already used functions and arguments on the previous slides.

≡ Functions and Arguments (2)

- `print()` is a function which prints something to the console.
- The empty parentheses is where you pass an argument.
- An argument is information to help a function.

```
print(name)
```



- The argument here is `name` or the object we want printed.

≡ Functions and Arguments (3)

- Let's try a different function and argument.

```
round(3.14159265359)
```



- This is a function which will round a number to the nearest whole number.

```
round(pi)
```



- You can also just use `pi`.

≡ Functions and Arguments (4)

- Some functions require multiple arguments.
- If we want to round `pi` to **two** decimal places, we need another argument.

```
round(pi, digits = 2)
```



- If you run the code above, you'll now get a slightly different answer because the additional argument `digits = 2` has changed how the function behaves. Rather than rounding to a whole number, it will now round it to two decimal places.

≡ Functions and Arguments (5)

- A function is something that contains a set of instructions that takes input, does something with it, and gives you an output.
- Functions tend to require quite specific arguments.
- One useful tip is to use `?` to learn about any function.
- Run the code below and check the bottom right panel for more information about the `round()` function.

```
?round()
```





Comments (1)

- You might be thinking “wow, **objects, environment, functions, arguments**, how will I remember what the code does?”
- Don’t panic because R allows you to add your own notes (or comments)!
- We use the hash symbol to tell R where our comments are.
- R will ignore everything after a hash.
- Look at the examples on the next slide and then amend your own code to add your own notes.



Comments (2)

```
date() #will return today's date  
round(pi, digits = 4) #will round pi to four decimal places.
```



- **Top Tip:** Always use comments so that when you come back to your code, you can understand it.



≡ Packages

- When you first open RStudio you will have access to “base R” functions.
 - Think of this as the basic subscription.
- However, throughout your data analysis journey you’ll need to make use of more specialist functions beyond base R that belong to packages.
 - Think of these packages as an “app” or an extra “add-on”.
- One essential package is the `tidyverse` package.



Install Packages

- To install a package use this code:

```
install.packages("tidyverse") # install tidyverse package
```



- **You do not need to do this on the campus PCs as Tidyverse is already installed.**
- Just like a phone app, you only need to install a package once on your device.

≡ Load Packages

- Installing a package is the first step. Next you need to load it.
- To load a package you'll use the `library()` function.
- You'll need to run this each time you start a new session in R.

```
library(tidyverse)
```



- If you take a look at the console you will see the package load.



Check Packages

- On the miscellaneous panel (bottom right) you'll notice a [Packages](#) tab.
- Click on this to see which packages are already installed on your device.
- If there is a tick next to the package name, this means it has been loaded and ready to use.

≡ Tidyverse



- Tidyverse is a package (or a collection of packages) which you'll use every week.
- It contains several essential functions to support with data analysis in Psychology.

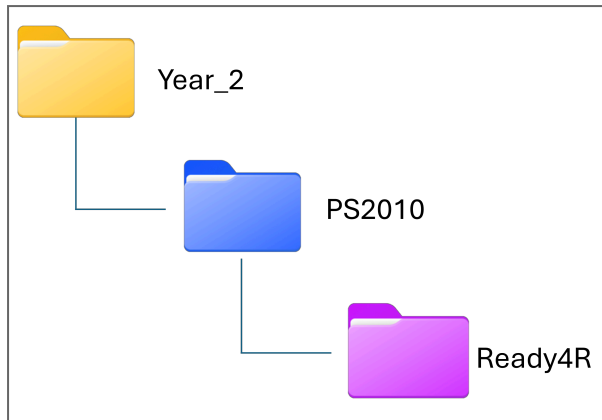
≡ Importing Data

- You will need to understand how to import and inspect different data files in RStudio.
- During PS2010 you'll be using `.csv` files, which are basically excel spreadsheets.
 - `.csv` stands for comma separated values and RStudio likes these files.
- There are a few different ways to get other types of files, learn more [**here**](#).
- During PS2010 you'll be expected to follow the steps on the next slides...
- But first...we need a data file.

≡ Files and Folder Structure (1)

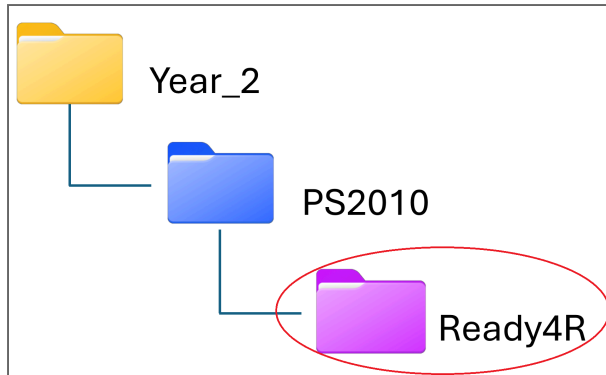
- Many previous PS2010 students have had difficulty with computer folder structures.
- Download the data file called [height.csv](#) from Moodle [here](#).
- Decide where to save it on your computer:
 - You might already have an existing file, e.g., Documents.
 - I'd recommend you create a new folder somewhere called "Ready4R" and save it there.

Files and Folder Structure (2)



- Saving your data in a folder that makes sense, such as “Ready4R”, will help you find your files next time.
- This will be especially important when you need to analyse your own data.
- For example: `Year_2 > PS2010 (subfolder) > Ready4R (subfolder)`.

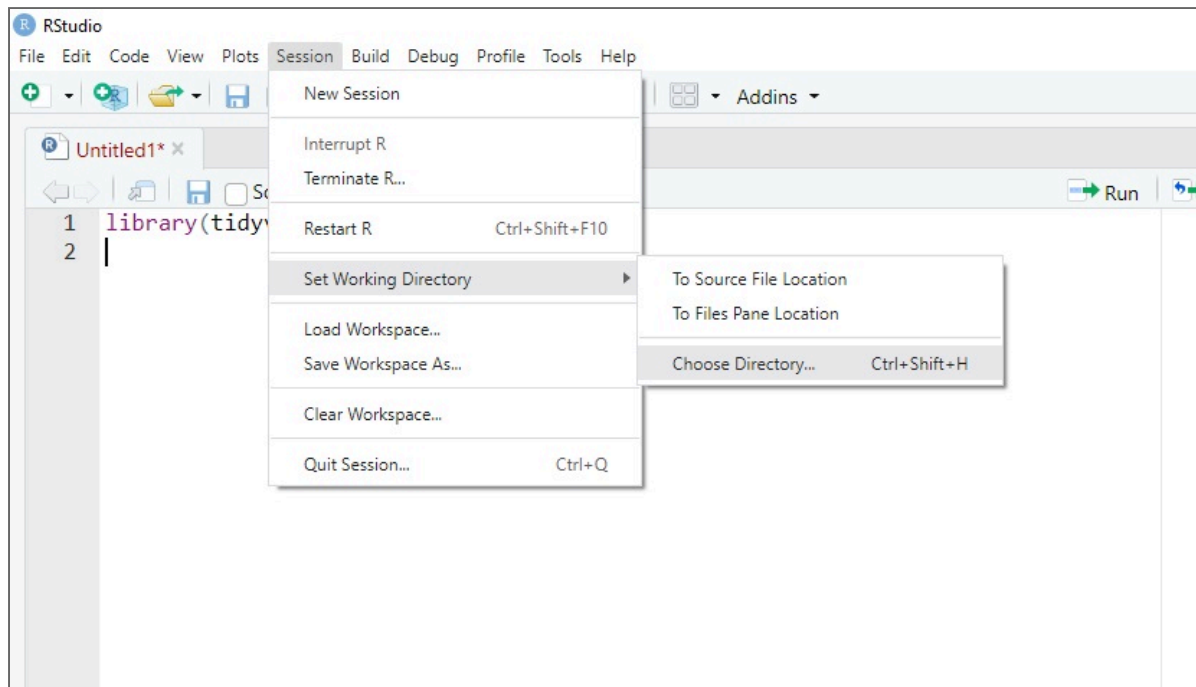
Working Directory



- The working directory is a folder on your computer which RStudio will use to find any data files.
- You'll need to "set" your working directory each time you use RStudio in order to tell it where to find your data file(s).
 - In this example, `Ready4R` would be your working directory, as it's where we saved our data `heights.csv`.

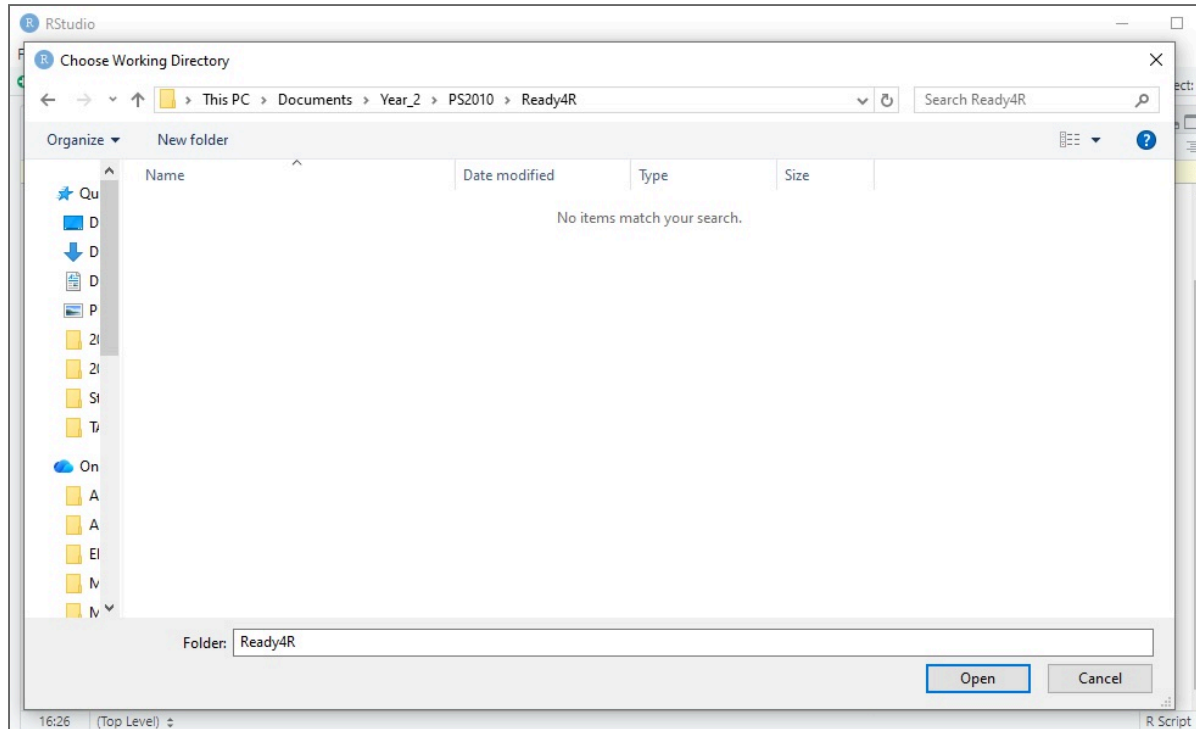
≡ Set Working Directory (1)

- Set the working directory now ([Session](#) > [Set Working Directory](#) > [Choose Directory](#)) :



≡ Set Working Directory (2)

- You need to select the **Folder** you saved the `height.csv` file in (e.g., "Ready4R").



≡ Check Working Directory

- Run the code below to ask RStudio where the current working directory is.
- You should check that this is correct and that it is the place your `height.csv` file is.

```
getwd()
```



≡ Import Data

- Now you will use your first tidyverse function: `read_csv()`

```
dat <- read_csv("height.csv")
```





read_csv() Explained

```
dat <- read_csv("height.csv")
```



- `dat` is an object. `Dat` is a short name for “data”. You could call it something else.
- `<-` is the assignment operator. Anything on the right side of the arrow will be assigned to the object on the left.
- `read_csv()` is a function which will read/look inside a `.csv` file
- `"height.csv"` is the argument which directs the function to a specific file name. Note it is in speech marks because it is a file name.

≡ Check the Environment

- You should now be able to check the **environment** panel.
- Remember from earlier that this panel is a bit like our warehouse where our boxes (or objects) are stored.
- You should see your newly created object called `dat`.
- If not, something has gone terribly wrong so put your hand up for some in-class help.

≡ View Data

- You can use the `view()` function to open up the object and view the raw data.

```
view(dat) # opens object to check raw data
```



≡ Summarise Data

- Finally, you can very quickly summarise the data using the `summary()` function.
- This will provide some (very) basic values within the data set.

```
summary(dat)
```



≡ What's Next

Thanks for taking part.

In this part of the workshop you have explored some R and RStudio basics.

This means you are now ready for the next stage...head back to the home screen to find the practical activity (part two of today's workshop).

You should work on this independently, but do not fear...we will be on hand to help you if you get stuck.

↓ **Download slides as PDF**